

LC MONITOR PROJECT HANDOFF GUIDE

1. Project Overview & Architecture

LC Monitor is an employee time-tracking, attendance, task management, and activity monitoring application. The system consists of three distinct parts:

1. **Frontend Dashboard:** A single-page application built with React, Vite, TypeScript, Tailwind CSS, and Shadcn UI. It handles authentication and shows real-time dashboards to employees, managers, and administrators. It is configured for static file hosting (e.g., cPanel).
 2. **PHP Same-Origin Proxy:** A server-side PHP proxy script hosted alongside the frontend. It intercepts dashboard requests and forwards them to the Supabase Edge Functions. This removes browser-side Cross-Origin Resource Sharing (CORS) blocks in production environments.
 3. **Backend & APIs (Supabase):** Consists of a Supabase PostgreSQL database, storage buckets for screen captures, and Deno Edge Functions that manage business logic (Authentication, Attendance, Tasks, Chats). Row Level Security (RLS) is active on screenshots and browser history, but disabled on base tables since authorization is managed inside the Edge Functions.
 4. **Client Data Collection (Chrome Extension):** A separate web extension installed on employee browsers. It takes screenshot captures, logs browser events, and authenticates securely using a secret UUID monitor token unique to each user profile.
-

2. Issues Solved & Actions Taken

1. cPanel Deployment & CORS Bypass

The Problem: The frontend is hosted on cPanel. Deno Edge Functions enforce strict CORS origins, which originally blocked direct dashboard API requests.

The Solution:

- We updated all local Deno Edge Functions (auth, admin, tasks, work-sessions, chat) to dynamically read origin headers and return wildcard access controls.
- Developed a PHP proxy script located at `public/supabase-proxy.php`. When deployed to cPanel, it intercepts dashboard API calls and performs server-side forwarding to Supabase, eliminating browser CORS blocks.
- Configured frontend API endpoints to automatically detect when they are running in production and route requests through this proxy.

2. Migration to the New Supabase Project (cuawkttwzfpjtwjaybu)

The target Supabase project reference has been updated to your live database `cuawkttwzfpjtwjaybu` (Project: `slm-employee-tracking`).

Files Modified:

- The local environmental file (.env): Swapped the project target references and added comments containing the old credentials for safekeeping.
- The Supabase CLI configuration file (supabase/config.toml): Linked the local workspace reference to the new project ID.
- The production PHP proxy (public/supabase-proxy.php): Updated the endpoint target URL.

3. Browser History Tracking (Feature Implementation)

We have fully built the database structure, Edge API routing, and frontend UI for the new Browser History feature.

- **Database Migration:** Created a database migration script (supabase/migrations/20260529180000_create_browser_history_table.sql) to set up the browser_history table, performance indexes on timestamps and session references, and Row Level Security policies.
- **Backend Endpoints:** Updated the work-sessions Edge Function to handle storing visited pages (validating client JWTs, extracting domains, connecting entries to active session, and bulk inserting) and reading visited pages (restricting managers to team-only rows while allowing admins to query all records).
- **Dashboard UI:** Developed the BrowserHistoryPage.tsx interface, featuring statistics cards, a domain breakdown list with gradient progress bars, and a chronological timeline showing visited time, page titles, links, durations, and favicons.
- **Routing & Navigation:** Registered routes in App.tsx and added sidebar navigation in AppSidebar.tsx (accessible to Managers and Admins).

3. Pending Roadmap for the Next Developer

To get these changes deployed and live, the incoming developer must complete the following technical tasks:

Task 1: Supabase CLI Authentication & Linking

The developer needs to open their terminal in the project directory, log in to Supabase CLI, and run the link command using your project reference cuawkttwzfpjtwjaybu. During this process, they will be prompted to enter the database password established when the Supabase project was created. If the password was forgotten, they must reset it via the Supabase Dashboard under Settings > Database.

Task 2: Pushing the Entire Database Schema

Since the new Supabase project is completely empty, the developer must push the local database migrations. This command will execute all migration files in order, creating the tables for users, teams, work sessions, breaks, screenshots, tasks, and our new browser history table on the live database.

Task 3: Edge Function Secret Settings

The Edge Functions require local environment secrets to validate employee session tokens. The developer must retrieve the project's JWT Secret from the Supabase Dashboard under Settings > API, and run the Supabase CLI secret setting command to register the JWT_SECRET key on the live project.

Task 4: Deploying Deno Edge Functions

The developer must deploy the updated work-sessions Edge Function folder from the local workspace to the live Supabase project to activate the new browser history GET and POST API endpoints.

Task 5: Client Web App Environment Key Setup

The developer needs to copy the publishable anon API key from the Supabase Dashboard (Settings > API) and paste it into the local .env file replacing the VITE_SUPABASE_PUBLISHABLE_KEY placeholder.

Task 6: Modifying the Chrome Extension (Data Harvester)

Once the extension source code is obtained from the original developer, the following updates must be made to capture browser data:

1. **Manifest File Permissions:** Modify the manifest file permissions array to request access to the browser history, tab APIs, activeTab, storage, and background alarms.
2. **Active Domain & Tab Focus Duration Tracking:** In the background scripts, write event listeners to track tab activated changes and window focus shifts. Use a local timer state to track the active tab's elapsed duration in seconds and extract clean hostname domains (e.g., parsing URL details to store only the root domain name).
3. **Batching and API Uploads:** Store captured browsing data in local storage temporarily. Set up an alarm to fire every few minutes to pull the stored visits, format them into an array containing the visited URL, tab page title, duration spent in seconds, ISO timestamp, and active session reference, and perform an HTTP POST request to the Edge Function browser-history endpoint. Send the employee's monitor token in the Authorization header. Once the server returns a successful response, clear those items from local storage.
4. **15-Minute Screenshots Timer:** Find the screenshot capture loop inside the background extension scripts and update the alarm trigger rate to capture and upload screen captures exactly every 15 minutes.